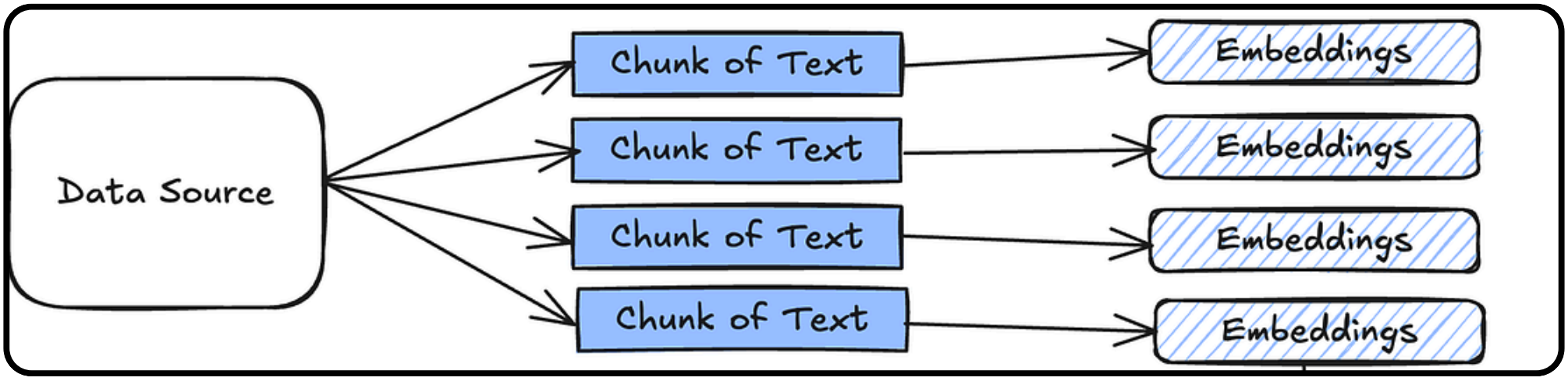
Chunking Strategies That Actually Work for RAG

Most RAG systems don’t fail because of the LLM.
They fail because of bad chunking.

How you split documents directly determines what the model can retrieve and what it will miss.



Let’s break down 3 chunking strategies that actually work, with LangChain code examples

Saleamlak Asrat Sheferie

1) Fixed-Size Chunking

Fixed-size chunking splits text into chunks of a predefined length.
Each chunk has the same size, usually measured in characters or tokens.
This is the simplest and most commonly used chunking strategy.

```
from langchain.text_splitter import CharacterTextSplitter

text_splitter = CharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = text_splitter.split_text(document)
```

Best for

Best for simple documents with uniform structure.
Works well for small datasets and quick prototypes.
Not ideal when document sections vary in length or meaning.

2: Recursive Chunking

What it is:

Recursive chunking splits text using a hierarchy of separators.
It tries larger structural boundaries first, then smaller ones if needed.
This preserves context better than fixed-size chunking.

```
from langchain.text_splitter import RecursiveCharacterTextSplitter

text_splitter = RecursiveCharacterTextSplitter(
    chunk_size=500,
    chunk_overlap=50
)

chunks = text_splitter.split_text(document)
```

Good for

Ideal for structured documents like articles and reports.
Helps maintain semantic boundaries between sections.
A strong default choice for most RAG systems.

@Saleamlak Asrat Sheferie

3: Semantic Chunking

What it is

Semantic chunking groups text based on meaning, not length.

Chunks are created by detecting topic or semantic shifts.

This produces more meaningful retrieval units.

```
from langchain_experimental.text_splitter import
SemanticChunker
from langchain.embeddings import OpenAIEmbeddings

text_splitter = SemanticChunker(
    OpenAIEmbeddings()
)

chunks = text_splitter.split_text(document)
```

Best for

Best for long, complex documents with multiple topics.

Improves retrieval relevance for knowledge-heavy RAG systems.

More expensive, but higher quality.

Why Chunking Matters in RAG

Chunking decides **what gets retrieved.**
Retrieval decides **what the LLM sees.**
And the LLM can only answer based on that context.

Bad chunking → irrelevant context → weak answers.
Good chunking → precise retrieval → reliable RAG.

Chunking is not preprocessing —
it's **system design.**

@Saleamlak Asrat Sheferie

@Saleamlak Asrat Sheferie

If you found valuable

Comment

Share. to Others

Repost

And Save for later

Follow Saleamlak Asrat Shiferie

for more content like this